

DRAW THE CHART

Build Handoff & Technical Specification

Everything built, everything left, and the full scoring internals

Internal build document · Version 1.0 · June 2026

How to use this document. This is a self-contained brief for whoever builds the next version of Draw The Chart — a human engineer or an AI coding agent. It states the current state of the project, specifies the scoring engine precisely enough to re-implement it, lists what remains to be done in priority order, and explains how to rebuild the front-end cleanly while preserving the parts that matter. If you are an AI agent picking this up: read Sections 3 and 4 in full before writing any code, and treat Section 7 as hard constraints.

1. What the project is

Draw The Chart (DTC) is a house-versus-player crypto prediction game. The player sees the BTC price chart up to a frozen moment (the *anchor*), then **draws the future price path** they expect over a fixed horizon (15 minutes, 1 hour, 6 hours, 24 hours, or 7 days), stakes an amount, and is paid based on how good the prediction was. It sits between prediction markets (too simple — just yes/no) and leveraged trading (too complex — liquidations, sizing, execution). The chart is the game mechanic, not a decoration.

The product is a web app: a React front-end, a pure-TypeScript scoring engine, and a thin backend (currently Supabase for storing rounds; on-chain settlement is future work). It is deployed on Vercel and gated behind a password for friends-only testing.

2. Current state — what exists today

The project is past prototype. The core loop runs end to end in a sandbox against real historical BTC data. The list below is the honest status of each piece.

Area	Status	Notes
Scoring engine (v3)	DONE + validated	The new field-relative engine. Pure logic, fully tested against 5 years of data. The crown jewel.
Scoring engine (v0.2, old)	DEPRECATED	The original absolute-similarity engine. Still wired in for side-by-side comparison; to be deleted.
Drawing interaction	DONE	Freehand draw on chart, mobile touch, normalization into clean points. Good and reusable.
Game loop (sandbox)	DONE	Draw ☐ submit ☐ reveal ☐ score ☐ payout, on historical data, with replay animation.
Reveal / score display	PARTIAL	Shows the score and breakdown, but does not yet explain the field ranking. Highest-impact gap.
Front-end design / pages	ROUGH	Landing, FAQ, leaderboard, admin pages exist but are inconsistent. Intended for a clean rebuild.
Persistence (Supabase)	PARTIAL	Stores rounds + feedback. Needs v3 fields (percentile, seed, field version) added.
Wallet / real money	NOT STARTED	No wallet login, no on-chain settlement, no bankroll. All future work.
Deployment	DONE	Vercel auto-deploys from the master branch. Password-gated.

In one line: the hard math (fair + solvent scoring) is solved and proven; what remains is making the game *feel* right to players, then adding real-money infrastructure.

3. Repository map — keep vs. discard

The architecture deliberately separates the scoring “brain” from the UI “face.” The brain (~630 lines) imports nothing from React or the browser; it is pure functions. This is why the entire front-end can be rebuilt while the engine is reused untouched.

Keep (irreplaceable):

Path	What it is	Why keep
src/scoring/v3/	The v3 scoring engine: similarity.ts, field.ts, payout.ts, index.ts	The core IP. Hard to recreate; fully portable.
src/scoring/score.ts	Shared helpers (log-returns, resampling)	Used by the engine.

Path	What it is	Why keep
scripts/v3Probe.ts	Validation harness (fairness + economics)	Proves the engine works; re-run after any change.
scripts/fairnessProbe.ts	The original probe that found the old bug	Reference / regression check.
data/ (local only)	2.7M BTC 1-minute candles, 2021–2026	Needed for all testing. NOT on GitHub — back it up.
WHITEPAPER_v3.md + this PDF	Design spec and blueprint	Tells the rebuild how it should work.
drawing-capture logic	Inside DrawingChart.tsx: stroke ☐ normalized points	Fiddly to recreate; extract before scrapping the component.

Discard freely (rebuild cleanly):

All page layouts and styling, the navigation, the score/payout panels, and the Landing / FAQ / Leaderboard / Admin pages. None of it is precious. The look is the easy part to redo once the engine is preserved. The old v0.2 scoring files (everything in src/scoring *outside* v3, except the shared helpers) can also go once v3 is the only scorer.

4. The scoring engine (read this in full)

4.1 The core idea, and why the old engine was broken

The original engine graded a drawing on an **absolute** scale: it measured how close the drawing was to reality and mapped that to 0–100. This failed for a structural reason: how “close” a fixed-quality drawing looks depends entirely on the market that round. In a calm market, small wiggles look like big errors; in a wild market, a sloppy drawing looks fine. The result, measured on real data: a near-perfect drawing that was merely 8% late in time lost head-to-head to a *random scribble* in 6.9% of rounds. Players felt this as “random drawings score better than good ones.” It could not be fixed by tuning — the broken assumption was the absolute scale itself.

The v3 engine asks a **relative** question instead: “*given this exact round, did your drawing beat what a no-skill player would have drawn?*” It does this in three layers, each feeding the next:

Layer 1 – Similarity. Measure how close the drawing is to what actually happened. Output: a number 0–100. (Used only to *rank* — it is not the final score.)

Layer 2 – Field. Generate 5,000 fake “no-skill” forecasts for this exact round, score each with Layer 1, and rank the player among them. Output: a percentile, e.g. 0.77.

Layer 3 – Payout. Convert the percentile to a money multiplier with a published curve whose average payout is mathematically locked to 1 – house edge.

Why this is fair and solvent *by construction*: the 5,000 fakes face the same market the player does, so round difficulty cancels out automatically — a given drawing quality earns a similar percentile whether the market was calm or chaotic. And because a no-skill player is, by definition, indistinguishable from the fakes, their percentile is random (uniform), so their average payout is exactly 1 – house edge. The house edge is a design constant, not a lucky outcome of tuning.

4.2 Layer 1 — Similarity (0–100)

All paths are first converted to **log-return space** (each point becomes $\ln(\text{price} / \text{start price})$, so the path starts at 0) and resampled to **N = 120** evenly spaced points. All errors are scaled by an **effective volatility** so the score is read in units of how much the market actually moved:

$$\sigma_{eff} = \max(\text{std}(\text{actual path}), 0.5 \times \text{typical sigma for this timeframe})$$

Similarity is the sum of three components. Each measures one distinct thing, and no error is penalized twice (a deliberate fix — the old engine double-counted):

Shape & Timing (0–50) — the heaviest, because shape matters most. Uses **banded Dynamic Time Warping (DTW)**: it lines up the two paths allowing small timing shifts (up to 10% of the horizon, a band of 12 of the 120 points), then measures the leftover distance. This is the formal version of what the human eye does. A drawing with the right shape but slightly mistimed loses credit *smoothly* — there are no cliffs, which is what made the old turning-point component feel random.

$$S1 = 50 \times \exp(-\lambda_{\text{shape}} \times D / (N \times \sigma_{eff})), \lambda_{\text{shape}} = 2.2$$

Direction (0–30) — did the player get the up/down moves right? The horizon is split into halves, quarters, eighths; in each piece the sign of the predicted net move is compared to reality. Crucially each piece is **weighted by how much the market actually moved**, so calling the direction right on a big move counts far more than on a flat stretch (this kills the coin-flip noise the old engine had).

Level (0–20) — was the price level roughly right? Combines overall bias (was the whole drawing too high or too low) and endpoint error (where it finished), each scaled by σ_{eff} and passed through a smooth exponential ($\lambda_{\text{level}} = 1.2$).

A separate turning-point detector (Hungarian matching) is kept **only as a reveal-screen explainer** (“you called this top 9 minutes early”). It annotates; it does not affect the score or the money. This is intentional.

4.3 Layer 2 — The synthetic field

For each round, generate **B = 5,000** baseline forecasts from a fixed recipe, score every one with Layer 1 against the same realized path, and compute the player’s mid-rank percentile $U = (\text{count below player} + 0.5 \times \text{ties}) / B$. The field is the set of **every strategy a player with no real insight could use** — so playing any of them earns a roughly average percentile and therefore exactly the house edge. The mixture:

Share	Generator	What it represents
40%	Block bootstrap	Real recent price wiggles stitched together — realistic noise & fat tails
20%	Random walk (GBM)	Pure randomness at the current volatility level
15%	Trend extrapolators	“It has been going up, so it keeps going up” (several strengths)
10%	Mean reverters	“It has moved a lot, so it pulls back”
10%	Flat & drift lines	“Not much will happen” ± small slopes
5%	Smoothed / lagged replays	Plausible but low-effort shapes

Two engineering details that matter:

- **Deterministic seeding.** The 5,000 paths are generated from a seed derived from the round’s commitment (a hash fixed *before* the player draws). This means the house cannot secretly pick a friendlier field, and anyone can re-generate the exact field afterward to verify the result. The settlement is reproducible by a third party.
- **Antithetic mirroring.** The random generators (bootstrap, GBM) emit each path together with its exact mirror image. This guarantees the random part of the field can never accidentally lean bullish or bearish in a given round — it removes a subtle source of per-round unfairness.

4.4 Layer 3 — Payout

The percentile maps to a multiplier through a three-zone curve, published before each round:

- **Refund zone** (below the break-even percentile): partial loss, softened by a floor.

- **Profit zone** (above break-even): exponential growth — strong forecasts pay well.
- **Jackpot tail** (top ~1%): a linear ramp up to the cap for beating essentially the whole field.

The growth rate g is **solved numerically** so the average multiplier over all percentiles equals exactly $1 - h$. The recommended (“Standard”) profile and its payout table:

Parameter	Value	Percentile	Multiplier (per \$100)
House edge h	4%	0.50	$0.52 \times (\$52)$
Break-even percentile	0.80	0.65	$0.74 \times (\$74)$
Floor multiplier	$0.15 \times$	0.80 (break-even)	$1.00 \times (\$100)$
Refund exponent	1.8	0.90	$2.48 \times (\$248)$
Jackpot starts	0.99	0.95	$3.91 \times (\$391)$
Cap	$12 \times$	0.99	$5.63 \times (\$563)$
Solved growth g	9.0944	1.00 (beat all)	$12.00 \times (\$1,200)$

Score scale note for the UI: displayed score = percentile $\times$ 100. So 50 is an average no-skill result, **80 is break-even**, and 99+ is jackpot territory. This is a tougher-looking scale than the old engine’s, but it is honest — a clearly wrong prediction should score low, not 40.

4.5 Proof it works (validation results)

Run with: `npx tsx scripts/v3Probe.ts 240` (1,200 historical rounds across all timeframes). Latest results:

Test	Target	Result
Random scribble beats a good (8%-late) drawing	$< 1\%$ of rounds	0.0% (old engine: 6.9%)
Random walk — average payout	≤ 0.96 (no edge gained)	0.92
Trend-following — average payout	≤ 0.96	0.96 (exactly the edge)
Good drawings (lagged / damped / warped copies)	rank near top	beat $\geq 99\%$ of field
Perfect / near-perfect drawings	top of field	100th percentile

Interpretation: good drawings reliably win, no zero-skill strategy can beat the house edge, and the headline fairness bug is gone. One known and intentional behavior: a perfectly flat “no-thesis” line ranks low (around the 24th percentile), because most of the field expresses some view — the game rewards having one.

4.6 Full parameter sheet (current defaults)

```

LAYER 1 Similarity
  N (resample points)          120
  DTW band                     12 samples (= 10% of horizon)
  lambda_shape                 2.2
  direction levels / decay     3 / 2.5
  lambda_level                 1.2
  weights shape/dir/level     50 / 30 / 20
  sigma_eff floor              0.5 * typical-sigma(timeframe)

LAYER 2 Field
  B (field size)               5000 (client) / 20000-50000 (server, later)
  mixture                      40 / 20 / 15 / 10 / 10 / 5 (see 4.3)
  bootstrap block length       horizon / 8
  EWMA volatility lambda       0.94
  antithetic                   on, for bootstrap + GBM cohorts
  seed                         SHA256(round commitment) – reproducible

LAYER 3 Payout (Standard profile)
  house edge h                 0.04
  break-even percentile        0.80
  min multiplier (floor)       0.15x
  refund exponent              1.8
  jackpot start                0.99
  max multiplier (cap)         12x
  growth rate g                9.0944 (solved so avg payout = 1 - h)
  stake cap                    0.56% of bankroll (quarter-Kelly)

```

5. How to call the engine (the API)

The whole engine is reached through one function. A new UI only needs this:

```

import { scoreRoundV3 } from 'src/scoring/v3';

const result = scoreRoundV3({
  predictedPrices, // number[] : the player's drawing as prices, anchor first
  actualPrices,    // number[] : what really happened, anchor first, same length
  lookbackPrices,  // number[] : prices before the anchor (used to build the field)
  seed,            // number   : round seed (live: derived from commitment hash)
  stake,           // number   : optional, default 100
});

// result contains:
// similarity { shape, direction, level, total } // Layer 1, 0-100
// percentile 0..1 -> displayed score = percentile * 100
// beaten     how many of the field the player beat
// fieldSize  5000
// multiplier  payout multiplier
// profit     stake * (multiplier - 1)
// fieldSimilarities number[] // every field score, for the swarm visualization

```

It is a pure function: same inputs always give the same outputs, and it runs anywhere (browser or server) in about 100 milliseconds. Nothing else in the engine needs to be touched to build a new front-end.

6. What is left to do (in order)

We are in **Phase 3 of 5: make the game feel right**. The engine (Phase 1) and the basic playable loop (Phase 2) are done. The remaining work, in the order it should be tackled:

Step 1 — Lock fairness into automatic tests

Today the fairness proof lives in a script run by hand. Port its checks into the automated test suite so any future change that breaks fairness or the house edge fails the build immediately. *Done when:* the test suite asserts “no zero-skill strategy beats the edge” and “good drawings beat random > 99% of the time.” Effort: ~1 day.

Step 2 — Make v3 the only score

The reveal screen currently shows two scores (old + new) for comparison. Remove the old one and its payout curve everywhere. Keep the turning-point detector only as an annotation. Fix all labels to the new scale (break-even = 80). *Done when:* one score, one payout, no contradictory numbers on screen. Effort: ~1 day.

Step 3 — Make the reveal screen explain the ranking (highest impact)

This is the most important remaining piece. Players need to *see* why they ranked where they did, or the percentile feels arbitrary. Build: the field of 5,000 forecasts drawn faintly behind the chart (a “swarm”), a context line such as “field median accuracy 24/100 — almost nobody saw this move,” a tag for tail-event rounds, and the ability to inspect the few forecasts that just beat the player. The engine already returns the field scores needed for this. *Done when:* a confused tester can look at the reveal and understand who beat them and why. Effort: ~1 week.

Step 4 — Rebuild the front-end cleanly & re-test with friends

Rebuild the rough pages into a clean, consistent UI (see Section 8 for how). Add the v3 fields to storage (percentile, seed, field version). Point the leaderboard at percentile — because it is regime-fair, a leaderboard finally means something. Re-open to friends and compare fairness feedback against the old data already collected. *Done when:* more testers say “that felt fair” than before. Effort: 1–2 weeks of build + testing time.

Step 5 — Real-money infrastructure (only after Step 4 confirms it feels right)

The largest chunk: wallet login, stablecoin betting on Base, a bankroll vault with the stake and exposure caps from the spec, the commit-reveal settlement that makes rounds verifiable, a price oracle (multi-venue median), and legal / responsible-use controls (geo-gating, limits, self-exclusion). The server runs the same engine function at a larger field size (20k–50k). *Done when:* a round can be played for real money and independently verified. This is where most future effort goes, and none of it should start before the game is trusted.

Explicitly NOT now

Tokens, assets beyond BTC, player-vs-player mode, and early cash-out. All tempting, all distractions until the core is trusted. They become easier later and fix nothing now.

7. Hard constraints (do not violate)

These are the rules that keep the system fair and solvent. They were learned the hard way from the old engine. An AI agent should treat them as non-negotiable.

Do	Don't
Keep scoring a pure function, separate from UI	Mix scoring logic into React components
Let the money flow through the percentile rank	Pay out directly off the raw 0–100 similarity
Make every penalty smooth (no thresholds)	Add hard cutoffs / cliffs (“within 5% or zero”)
Penalize each error type exactly once	Double-count (e.g. a late turn as both missed and fake)
Seed the field from the pre-draw commitment	Generate the field after seeing the outcome
Re-solve g whenever a payout parameter changes	Hand-pick a multiplier curve and hope the edge holds

Do	Don't
Re-run v3Probe after any scoring change	Ship a scoring change without re-validating
Keep all naive strategies inside the field	Leave a naive strategy out (it becomes exploitable)

The single most important invariant: the average payout to a no-skill player equals 1 – house edge, in every market regime, by construction. If a change could break that, it is wrong — no matter how good it looks. The economics live in Layers 2 and 3; Layer 1 can be tuned freely for how the score *feels*, because it only affects ranking, never the edge.

8. How to rebuild the front-end cleanly

The recommended path for a clean rebuild, given the engine is already portable:

- **Start a fresh project** (React + TypeScript + Vite, or your framework of choice).
- **Copy src/scoring/ in untouched.** It works on day one because it never depended on the old UI. Add the v3Probe script too, and confirm it passes.
- **Build the round flow** around the single `scoreRoundV3()` call: setup ☐ draw ☐ submit ☐ reveal. Reuse the drawing-capture logic extracted from the old `DrawingChart` component; rebuild the chart visuals fresh (lightweight-charts or similar).
- **Design the reveal screen first-class** (Section 6, Step 3) — it is the heart of the product, not an afterthought.
- **Keep pages minimal** to start: Play, and a simple How-It-Works. Add leaderboard / FAQ / admin later. Resist rebuilding everything at once.

Back up the data/ folder before anything else — it is 65MB of historical candles that exists only locally (it is excluded from GitHub on purpose) and is needed to run the validation. Everything else is already safe in the repository.

9. Quick glossary

Term	Plain meaning
Anchor	The frozen “now” moment the round starts from
Horizon	How far into the future the player predicts (15m ... 7d)
Log-return space	Prices re-expressed as growth from the start, so different price levels compare fairly
The field	5,000 fake no-skill forecasts the player is ranked against each round
Percentile (U)	Fraction of the field the player beat; the basis of the score and payout
House edge (h)	The house’s guaranteed average cut, 4% in the Standard profile
DTW	Dynamic Time Warping — compares two paths allowing small timing shifts
Antithetic	Pairing each random path with its mirror so the field can’t lean directionally
Commit-reveal	Locking the round secretly first, revealing after, so results are verifiable

End of handoff. The fuller public-facing design rationale lives in WHITEPAPER_v3.md; the engine source is in src/scoring/v3/; the proof is scripts/v3Probe.ts. Start with Sections 3, 4, and 7.